# Informe breve – Taller práctico de Pruebas de Software y QA

**Proyecto:** Mini-pipeline ETL de clientes (CSV → transformación → SQLite)

## 1. Objetivo y alcance

Construir y verificar un proceso ETL que lee clientes desde un archivo CSV, normaliza y valida sus correos electrónicos y almacena los registros en SQLite. Se aplican pruebas unitarias, pruebas de integración, medición de cobertura y revisión estática de seguridad y mantenibilidad.

## 2. Implementación

La solución se divide en tres módulos: **transform.py** normaliza correos y calcula totales con descuento; **pipeline.py** valida columnas y registros y carga la información en SQLite; **reportes.py** consulta clientes mediante parámetros SQL, ejecuta únicamente reglas permitidas y calcula totales. La separación permite probar las responsabilidades de forma aislada.

## 3. Pruebas y resultados obtenidos

La suite utiliza **pytest.mark.parametrize** para validar distintos datos y **pytest.raises** para comprobar errores esperados. Incluye validación de correos, límites de descuento, valores negativos, redondeo, carga CSV→SQLite, idempotencia, rechazo de filas inválidas, validación de columnas y resistencia a una entrada de inyección SQL.

| Medición | Resultado de la ejecución de preparación |
|---|---|
| Pruebas aprobadas | 27 |
| Pruebas fallidas | 0 |
| Cobertura del paquete etl | 100% (51 sentencias, 0 sin ejecutar) |
| Reporte XML | coverage.xml y evidencias/coverage.xml |
| Reporte JUnit | evidencias/junit.xml |

Los valores anteriores corresponden a una ejecución real de la suite durante la preparación del paquete. Se recomienda repetirla en el equipo de entrega y conservar esa salida como evidencia reproducible.

## 4. Hallazgos y correcciones de seguridad

| Hallazgo de la versión inicial | Riesgo | Corrección |
|---|---|---|
| Contraseña fija en el código | Exposición de credenciales | Leer DB_PASSWORD desde el entorno; no guardar secretos. |
| SQL construido con f-string | Inyección SQL | Usar consulta parametrizada con ?. |
| Uso de eval() | Ejecución de código arbitrario | Eliminar eval y permitir solo reglas explícitas. |
| except: y pass | Errores ocultos y diagnóstico difícil | Capturar excepciones específicas y comunicar el error. |

La prueba de inyección SQL utiliza el texto `' OR '1'='1` y verifica que se trate como dato, sin alterar la consulta.

## 5. Relación con la rúbrica de evaluación

| Criterio | Evidencia en el repositorio | Estado |
|---|---|---|
| Pruebas unitarias (25%) | tests/test_transform.py; evidencias/pytest_salida.txt y PNG | Ejecutado: 27 pruebas |
| Integración (25%) | etl/pipeline.py; tests/test_integracion.py; junit.xml | Ejecutado |
| Cobertura (15%) | coverage.xml; evidencias/coverage.xml; captura pytest-cov | 100% observado |
| SonarQube (25%) | sonar/antes/reportes.py; etl/reportes.py; configuración y scripts | Pendiente de ejecutar en instancia SonarQube real |
| Informe (10%) | INFORME.md e INFORME_ENTREGA.pdf | Preparado; completar métricas SonarQube |

## 6. Análisis SonarQube: procedimiento y limitación

El repositorio incluye una copia de reportes.py con defectos intencionales para el análisis inicial y el módulo corregido para el análisis final. SonarQube debe ejecutarse en una instancia local: iniciar el servicio con `docker compose up -d`, crear el proyecto con clave `taller-calidad`, generar un token privado y ejecutar el scanner primero con la versión vulnerable y después con la versión corregida. Las capturas de Overview, Issues, Security Hotspots y Quality Gate deben guardarse en `evidencias/`.

**Importante:** no se fabrican capturas ni resultados de SonarQube. Un pantallazo simulado no demostraría que el análisis se ejecutó ni que el Quality Gate fue aprobado. Los valores de bugs, vulnerabilidades, code smells y Quality Gate se deben registrar a partir de la instancia real.

## 7. Conclusión y reflexión

Las pruebas unitarias permiten revisar funciones aisladas y sus límites; las pruebas de integración verifican que CSV, transformaciones y SQLite trabajen conjuntamente. La idempotencia es esencial en ETL porque una misma carga puede repetirse sin crear duplicados. La cobertura ayuda a identificar código no ejecutado, pero no garantiza por sí sola la ausencia de defectos. El análisis estático complementa las pruebas al señalar riesgos de seguridad y mantenibilidad. La combinación de estas técnicas aumenta la confianza en el procesamiento y facilita mantener el sistema.